Maven : Project structure

POM; dependencies; lifecycle; profiles

Rahul Guha || SME || Java FS Training || HCL
4. Build Tools, IDE & Debugging

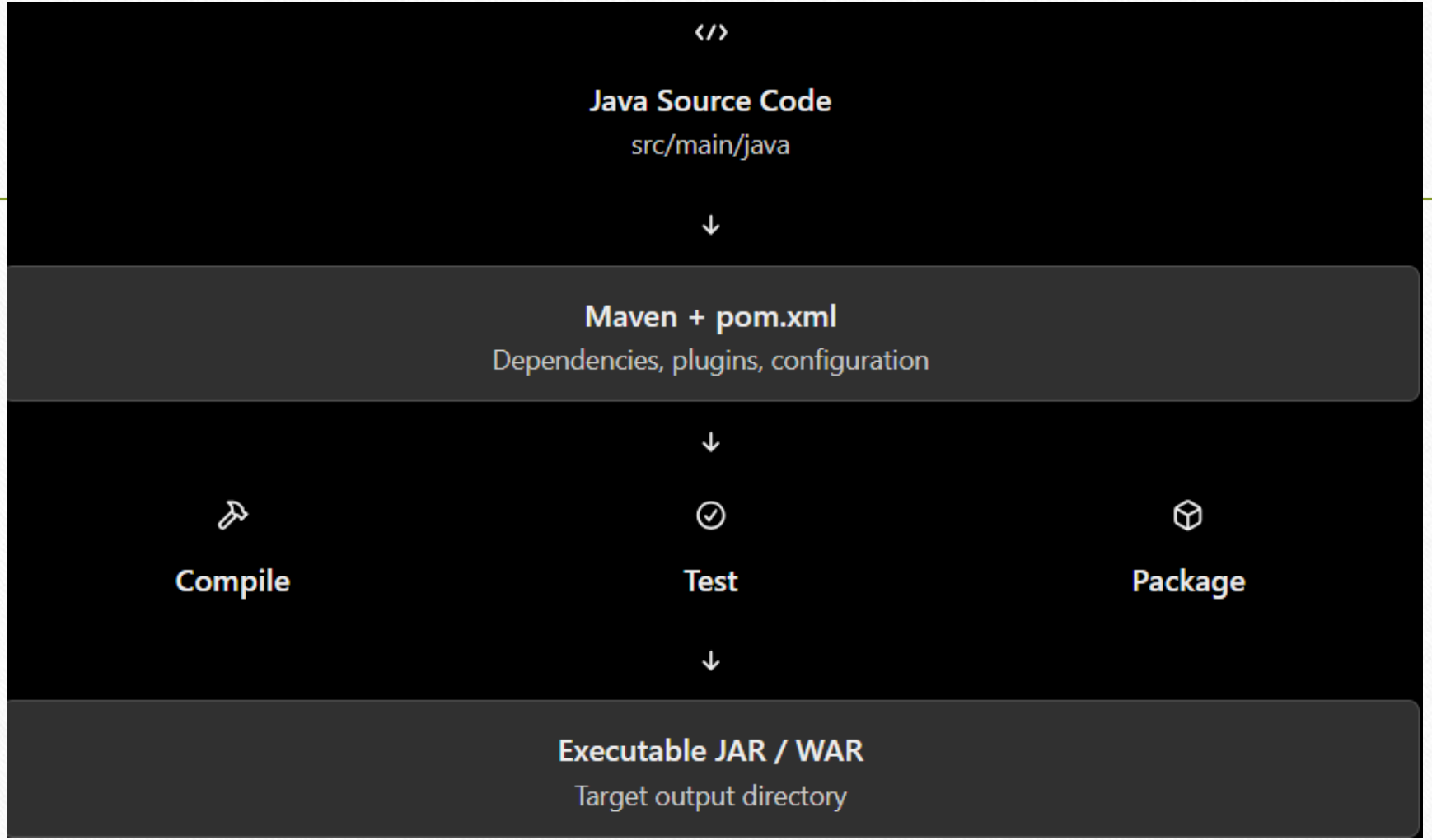
Maven in Java

Maven is a build automation and project management tool used in Java development. It simplifies dependency management, compilation, testing, packaging, and deployment.
These notes cover five major topics:
1. Maven project structure
2. POM (pom.xml)
3. Dependencies and dependency management
4. Maven build lifecycle5. Maven profiles

1. What is Maven?
Apache Maven is a build automation tool that uses an XML configuration file called pom.xml (Project Object Model).Without Maven, developers may need to download JAR files manually, configure classpaths, compile Java files, run tests, and create JAR files themselves.
Maven automates these tasks through standardized commands.

Why do we use Maven?-
-Automatically downloads and manages required libraries.
- Provides a standard project directory structure.
- Compiles and tests code using simple commands.
- Packages applications into JAR or WAR files.
- Supports different environments through profiles.
- Integrates with CI/CD tools such as Jenkins and GitHub Actions.

Maven workflow



2. Maven project structure

A standard Maven Java project follows this structure:

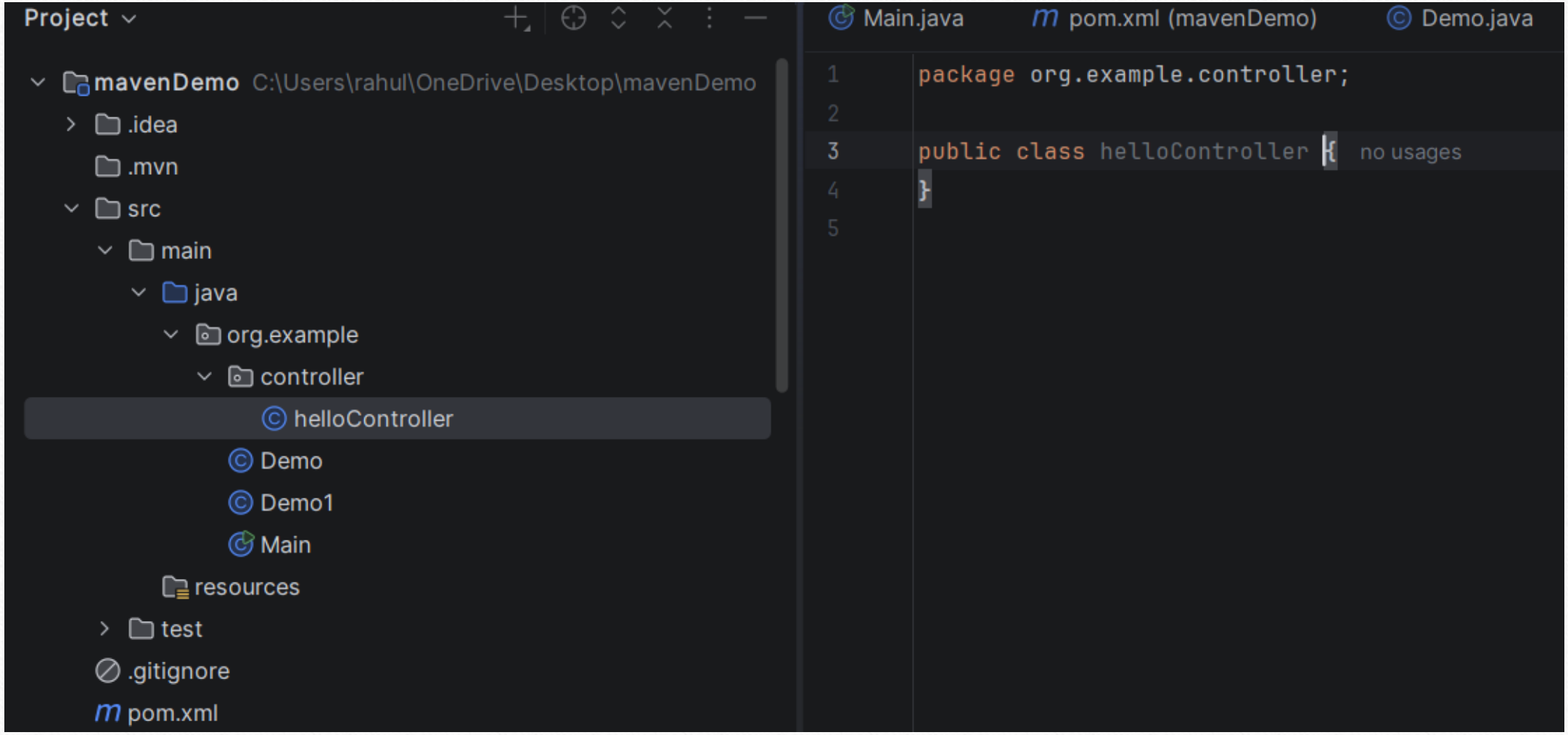
```
Plain text

MavenDemo/
│
├── pom.xml
├── src/
│   ├── main/
│   │   ├── java/
│   │   │   └── com/example/
│   │   │       └── App.java
│   │   └── resources/
│   │       └── application.properties
│   │
│   └── test/
│       ├── java/
│       │   └── com/example/
│       │       └── AppTest.java
│       └── resources/
│
└── target/
    ├── classes/
    ├── test-classes/
    └── MavenDemo-1.0-SNAPSHOT.jar
```

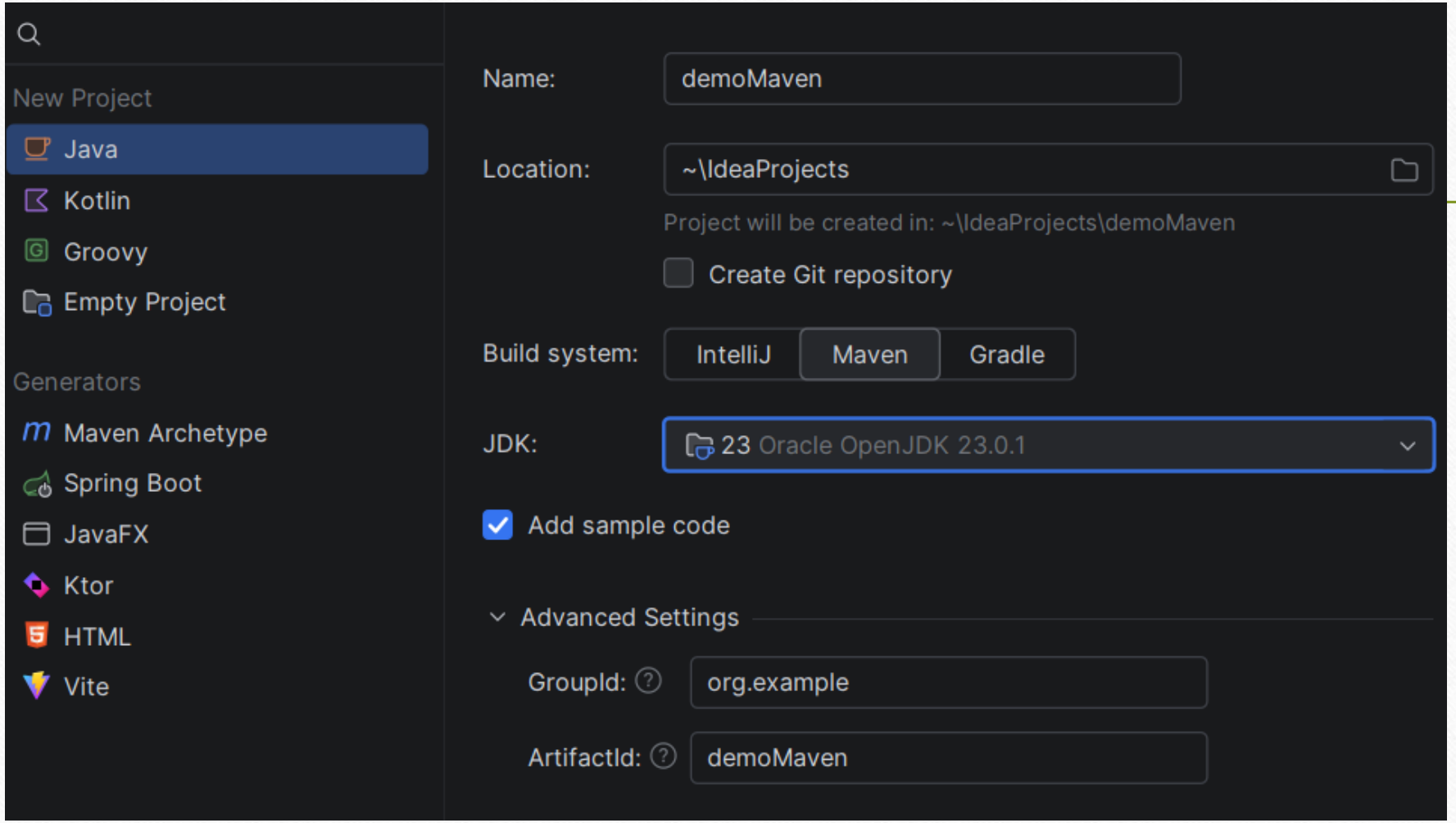
Purpose of each directory

File or directory	Purpose
`pom.xml`	Defines project settings, dependencies, plugins and profiles
`src/main/java`	Main Java source code
`src/main/resources`	Configuration files and other runtime resources
`src/test/java`	Test source code
`src/test/resources`	Resources used during testing
`target/classes`	Compiled application classes
`target/test-classes`	Compiled test classes
`target/`	Generated build output

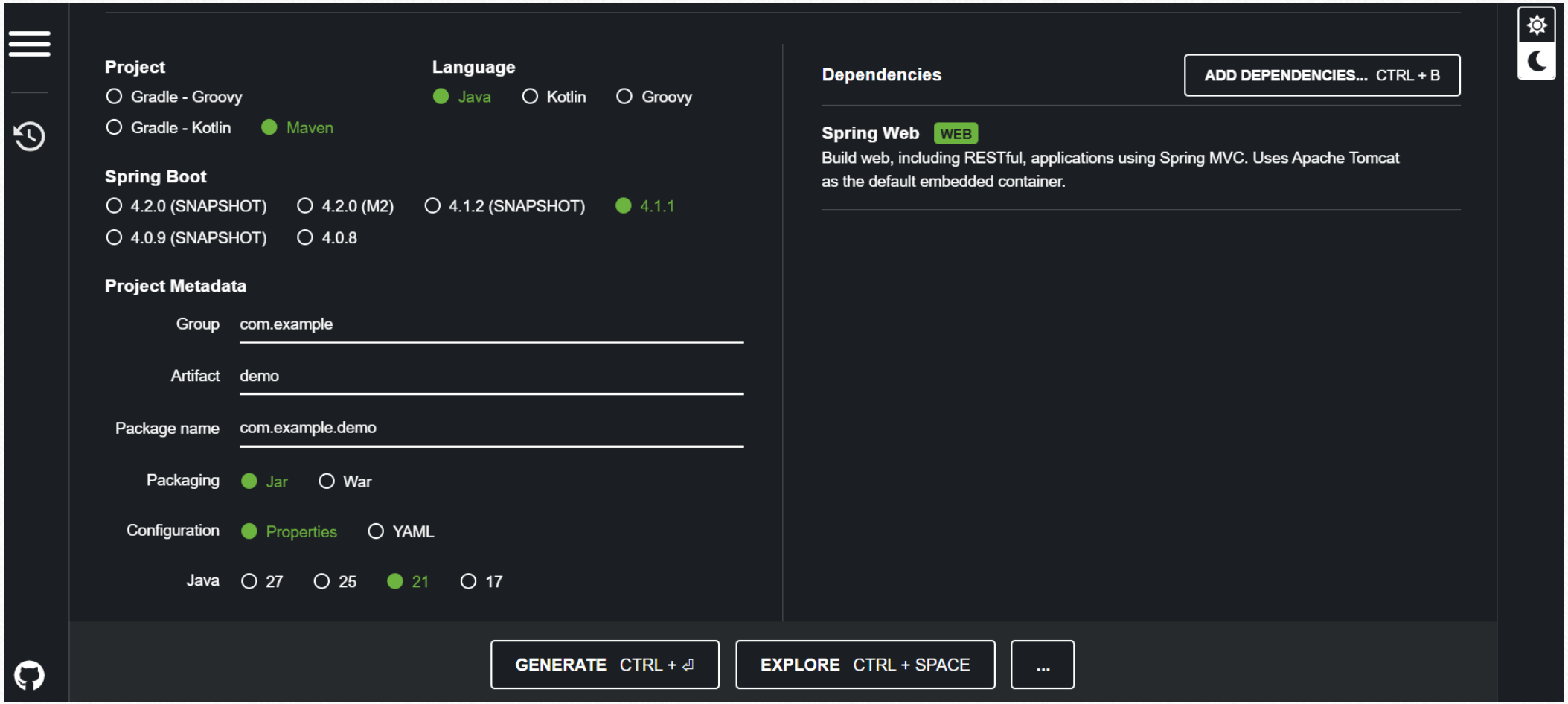
2. Maven project structure



2. Maven project structure



2. Maven project structure



2. Maven project structure

<abc></abc>

Agaist the Rule. To Write anything.

<modelVersion>4.0.0</modelVersion> - describes POM's Model Version

////////////////////////////////////

<groupId>org.example</groupId>

It Must be Unique. So we use Domain Name. It is maven oriented not spring Oriented.

<groupId>in.jecrc</groupId>

////////////////////////////////////

<artifactId>demoMaven</artifactId> Means Project Name

////////////////////////////////////

<version>1.0-SNAPSHOT</version> Snapshot are those who are in working stage.

////////////////////////////////////

```
<properties>
    <maven.compiler.source>23</maven.compiler.source>
    <maven.compiler.target>23</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
</properties>
```

works in Key Value Properties

2. Maven project structure

Now I can create my own property

```
 <properties>
        <Java.version>23</Java.version>
        <name>Rahul Guha</name>
        <maven.compiler.source>23</maven.compiler.source>
        <maven.compiler.target>23</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
```

Also can add name in version

```
<version>1.0-SNAPSHOT-Rahul_Guha</version>
<version>1.0-SNAPSHOT</version>
```

Another method to call author name

```
<version>1.0-SNAPSHOT-${author.name}}</version>
```

2. Maven project structure

Now there is another popular property called

<packaging>war</packaging>

<packaging>jar</packaging>

If you wan to download dependencies then its by default structure is:

```
<dependencies>
    <dependency>
      <groupId></groupId>
      <artifactId></artifactId>
       <version></version>
    </dependency>
  </dependencies>
```

to add database connectivity

I will add mysql connector.

To download External Dependencies go to: https://mvnrepository.com/

Then search mysql connector

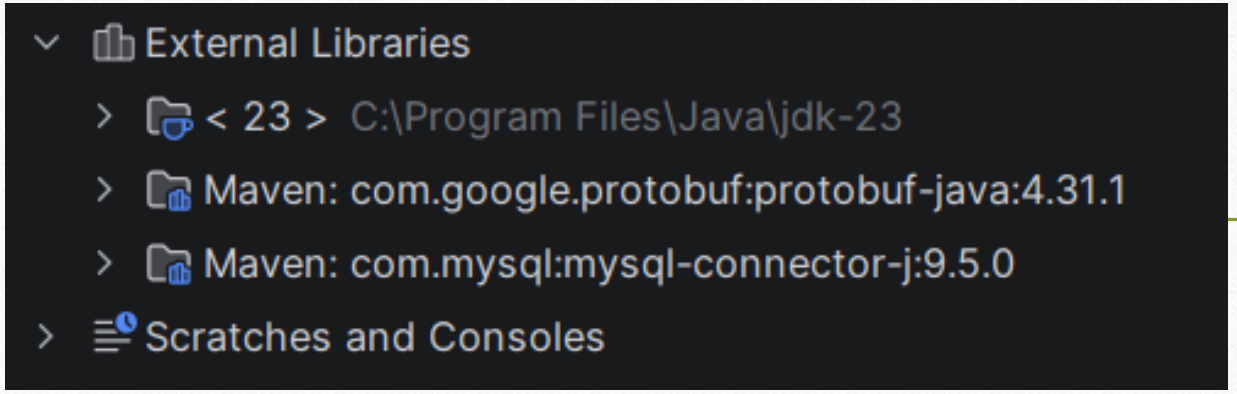
Not the latest one it might carry errors. I will choose

MySQL Connector/J » 9.5.0

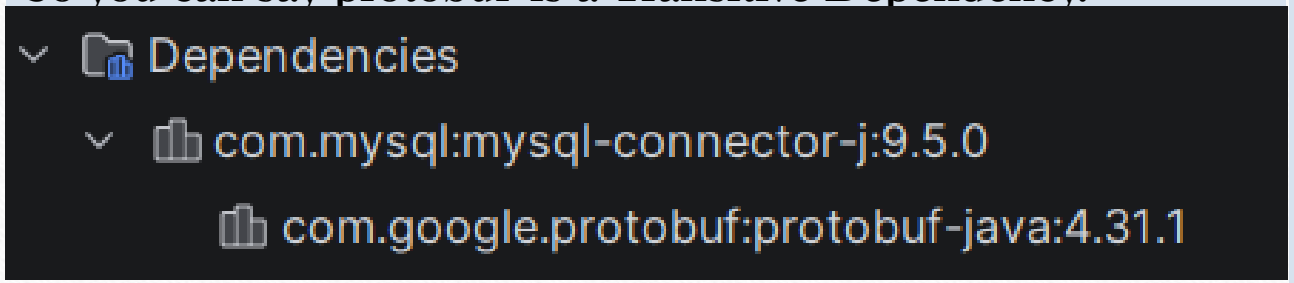
Only getting the dependency on the code is not enough to download. Go to external libraries you can't find them

After commanding to maven to get thise dependencies you can get it on external libraries.

2. Maven project structure



We have asked for mysql connector but I also get a JAR called Protobuf.
Now go to maven
-> You can see another folder called Dependencies.
So you can say protobuf is a Transitive Dependency.



Suppose now I have to work with Hibernet.(hibernate)
I will cohoose
-> Hibernate ORM Hibernate Core
I can see it will be installed with transitive dependencies.
Now Move to spring Initializer Project and find its pom.xml

There you can get a tag
<parent>

<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>4.1.1</version>
<relativePath/> <!-- lookup parent from repository -->
</parent>

2. Maven project structure

Every pom is inherited from parent POM.
When you create a class in jva and don't inherit any other class then by default you inherit object class.

Need for the <parent> tag in Maven

In Maven, the <parent> tag is used when one Maven project inherits configuration and dependencies from another Maven project. The main purpose is to avoid duplication and maintain consistent configuration across multiple projects.

1. Syntax of the <parent> tag

```
<parent>
    <groupId>com.example</groupId>
    <artifactId>my-parent</artifactId>
    <version>1.0.0</version>
</parent>
```

These three coordinates identify the parent Maven project:-

groupId: Organization or project group.
artifactId: Parent project's name.
version: Parent project's version.

The child inherits the parent's groupId, version, and compiler configuration. It does not need to repeat them.relativePath tells Maven where to look for the parent POM locally. Its default is ../pom.xml, so you can omit it when the parent is located there.

4. What can a child inherit from a parent?

Configuration	Inherited?
Properties	Yes
Dependencies in `<dependencies>`	Yes
Dependency versions in `<dependencyManagement>`	Available to children when they declare those dependencies
Plugins in `<build><plugins>`	Yes
Plugin versions in `<pluginManagement>`	Managed for child plugins that use them
`groupId` and `version`	Yes, if omitted in the child
`artifactId`	No; each child identifies its own artifact

2. Maven project structure

4. What can a child inherit from a parent?

Configuration	Inherited?
Properties	Yes
Dependencies in `<dependencies>`	Yes
Dependency versions in `<dependencyManagement>`	Available to children when they declare those dependencies
Plugins in `<build><plugins>`	Yes
Plugin versions in `<pluginManagement>`	Managed for child plugins that use them
`groupId` and `version`	Yes, if omitted in the child
`artifactId`	No; each child identifies its own artifact

2. Maven project structure

Difference between <parent> and <dependency>

`<parent>`	`<dependency>`
Inherits project configuration	Adds a library or module dependency
Shares properties, build configuration, and managed versions	Makes dependency classes available to the project
Identifies another POM	Identifies a library or artifact
Does not automatically add every parent dependency in every configuration scenario	Declares the dependency the project needs

Important: A parent POM and a Maven multi-module project are related but different concepts. A project can inherit from a parent without being a module of that parent. For a multi-module build, the parent typically also declares its modules using <modules>.
RememberUse <parent> when several projects should share common configuration, dependency management, or plugin settings. It reduces duplication and makes version and build management easier.

2. Maven project structure

In Maven, the <repository> tag specifies where Maven should look to download project dependencies (JAR files) and other artifacts that are not available in the repositories it normally searches. Maven uses Maven Central by default, so you generally need <repository> only when a dependency is hosted in another repository.

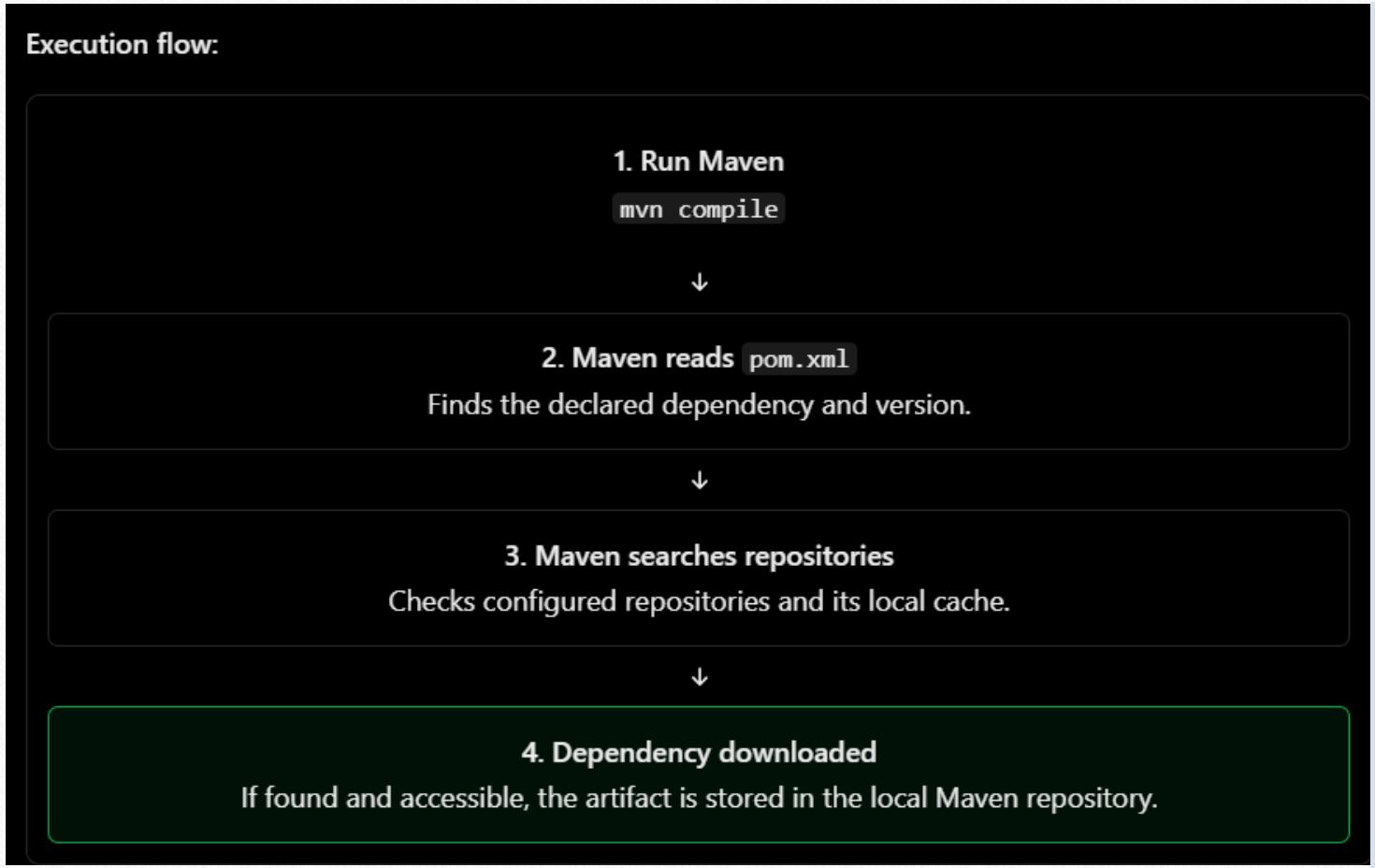
1. Syntax of <repository>You place the <repository> tag inside <repositories> in your project's pom.xml.

```
<repositories>
   <repository>
      <id>my-repository</id>
      <name>My Custom Repository</name>
      <url>https://example.com/maven-repository/</url>
   </repository>
</repositories>
<profile>
   <id>local-dev</id>
   <repositories>
      <repository>
         <id>internal-maven-repo</id>
         <name>Internal Repository</name>
         <url>https://host/internal-maven-packages</url>
      </repository>
      <repository>
         <id>central</id>
         <name>Central Repository</name>
         <url>https://repo.maven.apache.org/maven2</url>
      </repository>
   </repositories>
</profile>
```

Tag	Purpose
`<repositories>`	Contains one or more repository definitions.
`<repository>`	Defines an individual repository.
`<id>`	Unique identifier for the repository within the project configuration.
`<name>`	Human-readable repository name; optional.
`<url>`	Repository location; required.
`<releases>`	Configures whether release artifacts can be downloaded.
`<snapshots>`	Configures whether snapshot artifacts can be downloaded.

2. Maven project structure

Use of the <repository> tag in Maven




```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-
instance"

xsi:schemaLocation="http://maven.apache.org/POM/4.0.
0
        https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>
    <groupId>com.example</groupId>
    <artifactId>demo</artifactId>
    <version>1.0.0</version>
    <repositories>
        <repository>
            <id>custom-repo</id>
            <url>https://repo.example.com/maven2/</url>
        </repository>
    </repositories>

    <dependencies>
        <dependency>
            <groupId>com.example.library</groupId>
            <artifactId>my-library</artifactId>
            <version>1.0.0</version>
        </dependency>
    </dependencies>
</project>
```

2. Maven project structure

Difference between Super POM and Effective POM in Maven

In Maven, both Super POM and Effective POM are related to the configuration Maven uses to build a project, but they are not the same.- Super POM: The default parent POM provided by Maven.- Effective POM: The final, merged POM configuration Maven calculates for your project after combining inherited and applicable configuration.

1. Super POM

Every Maven project inherits default configuration from Maven's built-in Super POM, even if you do not explicitly write a <parent> tag in your pom.xml.

It provides default settings such as:-

- Default repositories and plugin repositories.

- Default build directory conventions.

- Default plugin configuration and other Maven conventions.

For example, your project might contain only:

```
<project>
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>demo</artifactId>
  <version>1.0.0</version>
</project>
```

Even though you have not configured every build setting, Maven still uses its built-in defaults through the Super POM.

2. Maven project structure

Difference between Super POM and Effective POM in Maven

2. Effective POM

The Effective POM is the complete configuration Maven derives for a specific project. It combines the project's pom.xml, inherited parent POM configuration, Maven's Super POM defaults, and other applicable configuration such as profiles.

For example, suppose your project defines:

```
<properties>
   <java.version>17</java.version>
</properties>
```

And its parent POM defines shared dependencies or plugin settings. The Effective POM shows the resulting configuration after Maven applies inheritance and merging rules.

How to generate the Effective POM

Run this command inside your Maven project directory:

mvn help:effective-pom

To save the result to a file:

mvn help:effective-pom -Doutput=effective-pom.xml

2. Maven project structure

3. Super POM vs Effective POM

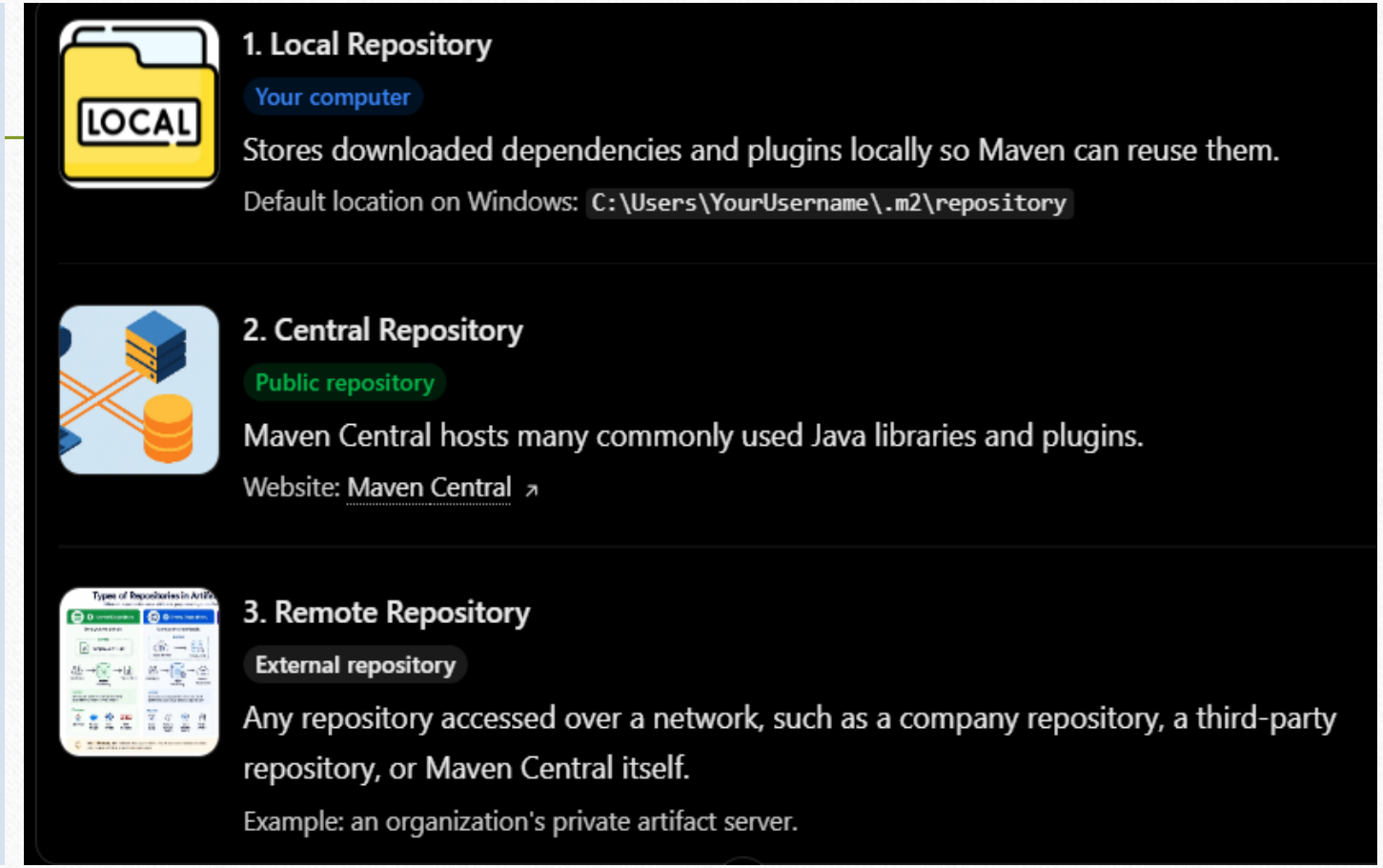
Feature	Super POM	Effective POM
Meaning	Maven's built-in default parent POM	Final calculated configuration for a project
Scope	Provides common defaults	Specific to the project and its active configuration
Contents	Default Maven configuration	Project settings, inherited settings, Super POM defaults, and applicable profile configuration
Customization	Supplied by Maven	Derived from the project's configuration
Explicitly created by developer?	No	Can be generated for inspection
Command	No command needed to use it	`mvn help:effective-pom`

2. Maven project structure

Maven Repositories

A Maven repository is a location where Maven stores and retrieves project artifacts, such as JAR files, POM files, libraries, and build plugins. For example, if your Java project uses the MySQL Connector/J library, Maven downloads the required JAR from a repository and makes it available to your project.

1. Types of Maven repositoriesMaven has three main repository types.



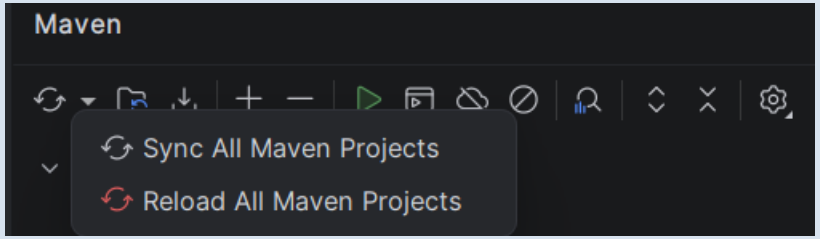
2. Maven project structure

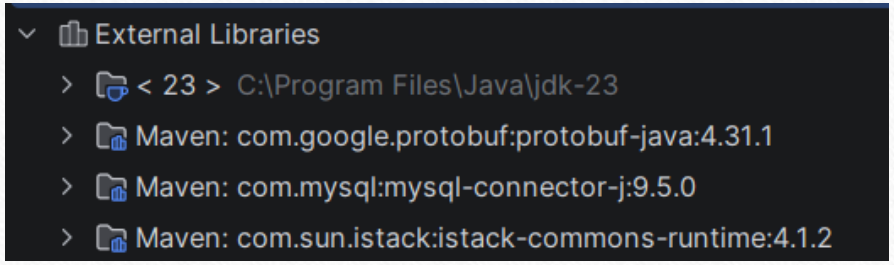
Maven Repositories

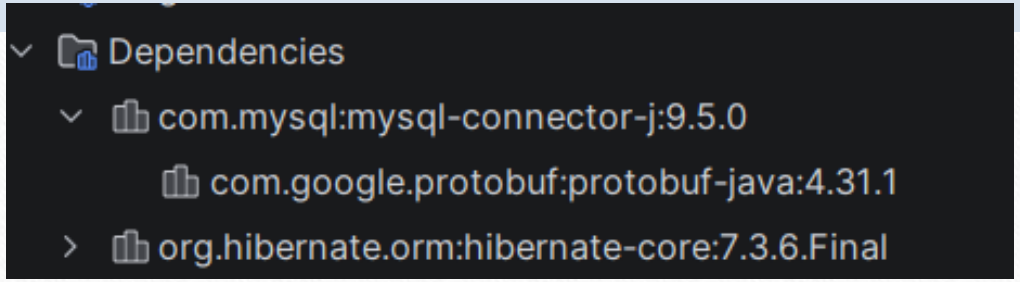
To change the repository source you can define in pom.xml and it will show to effective pom.
Like

```
<repositories>
    <repository>
        <id>Company Repo</id>
        <url>www.amazon.com/maven</url>
    </repository>
</repositories>
```

Now delete all Repositories and go to maven and reload all.
Also check in dependencies & External Libraries.







2. Maven project structure

Maven Lifecycle

Maven Lifecycle — Notes, Commands, Examples and Execution Flow

The Maven Lifecycle is a sequence of predefined phases used to automate the build process of a Java project, including compilation, testing, packaging, installation, and deployment.

For example, instead of compiling Java files and creating a JAR manually,

you can run: mvn package

Compile -> Validdate + Compile

1. Three built-in Maven lifecycles

Default Lifecycle

Builds the application: compile, test, package, install, and deploy.

Clean Lifecycle

Removes generated files from previous builds, usually the `target` directory.

Site Lifecycle

Generates project documentation and reports.

3. Important commands

Command	Purpose
`mvn validate`	Validate the project
`mvn compile`	Compile main source code
`mvn test`	Compile and run unit tests
`mvn package`	Build and package the application
`mvn verify`	Run verification checks
`mvn install`	Install the artifact in the local repository
`mvn deploy`	Publish the artifact to a configured remote repository
`mvn clean`	Remove generated build output
`mvn clean package`	Clean first, then build and package

2. Maven project structure

Maven Lifecycle

Lifecycle	Phase	One-line description
Clean	pre-clean	Executes tasks before cleaning the project.
Clean	clean	Removes generated files, usually the target directory.
Clean	post-clean	Executes tasks after cleaning.
Default	validate	Validates the project configuration.
Default	initialize	Initializes build properties and state.
Default	generate-sources	Generates source code for compilation.
Default	process-sources	Processes source files.
Default	generate-resources	Generates application resources.
Default	process-resources	Copies and processes resources into the output directory.
Default	compile	Compiles main Java source code.
Default	process-classes	Processes compiled main classes.
Default	generate-test-sources	Generates test source code.
Default	process-test-sources	Processes test source files.
Default	generate-test-resources	Generates test resources.
Default	process-test-resources	Copies and processes test resources.

2. Maven project structure

Maven Lifecycle

Lifecycle	Phase	One-line description
Default	test-compile	Compiles test source code.
Default	process-test-classes	Processes compiled test classes.
Default	test	Runs unit tests.
Default	prepare-package	Prepares files before packaging.
Default	package	Creates a JAR or WAR file.
Default	pre-integration-test	Prepares the integration-testing environment.
Default	integration-test	Runs integration tests.
Default	post-integration-test	Performs cleanup after integration tests.
Default	verify	Checks that the package meets verification requirements.
Default	install	Installs the artifact in the local Maven repository.
Default	deploy	Publishes the artifact to a remote repository.
Site	pre-site	Performs tasks before site generation.
Site	site	Generates project documentation and reports.
Site	post-site	Performs tasks after site generation.
Site	site-deploy	Publishes generated documentation to a configured location.